

Mapeamento de Entidades Oficiais do Projeto FashionAI

Especificação oficial | TCC 2026 · SAI · 17 de September de 2026

Objetivo: Mapear todas as entidades de banco de dados que serão implementadas no novo repositório git oficial do projeto FashionAI, considerando a arquitetura poliglota MySQL + NoSQL.

Entidade	Descrição	Banco de Dados	Tabela/Coleção	Status	Notas
User	Usuário do sistema (perfil, credenciais)	MySQL	users	Implementado	Fonte de verdade para autenticação e perfil
WardrobeItem (clothes_piece)	Peça individual do guarda-roupa	Firestore (NoSQL)	wardrobe_items (coleção)	Implementado	Armazenamento flexível com metadados de IA (modelo 3D, segmentação)
PieceMetadata	Metadados de análise de peça (dimensões, confiança)	Firestore (NoSQL)	Subdocumento em wardrobe_items	Implementado	Documento aninhado dentro de wardrobe_items
AllInference	Resultado de detecção de IA (categoria, cor, material)	Firestore (NoSQL)	Subdocumento em wardrobe_items	Implementado	Armazenado como AI detections dentro de wardrobe_items
PipelineJob	Job de processamento assíncrono (background removal, 3D)	Firestore (NoSQL)	pipeline_jobs (coleção)	Implementado	Rastreamento de status de processamento em tempo real
BackgroundRemovalJob	Job específico de remoção de fundo	Firestore (NoSQL)	Subcoleção em pipeline_jobs	Implementado	Especialização de PipelineJob para processamento de imagem
Scheme (clothes_scheme)	Look/Outfit - composição de peças no visual	Firestore (NoSQL)	schemes (coleção)	Implementado	Entidade raiz para o conceito de look/esquema
SchemeItem	Peça dentro de um esquema (slot, posição, zIndex)	Firestore (NoSQL)	Subcoleção scheme_items dentro de schemes	Implementado	Vínculo polimórfico: pode referenciar wardrobe_item sugerido ou índice de slot
SchemeBackground	Configuração visual do fundo do look	Firestore (NoSQL)	Subdocumento em schemes	Implementado	Cores, gradientes, imagens de fundo
Brand	Marca de roupas	MySQL	brands	Implementado	Referência de dados estruturados e mestre de marcas
Market	Mercado (estação × gênero)	MySQL	markets	Implementado	Dimensão de segmentação (seasonal + gender)
PieceLike	Curtida em uma peça	MySQL	piece_likes	Implementado	Relacionamento usuário × peça; suporta COUNT()

OutfitComment	Comentário em um look/outfit	MySQL	outfit_comments	Divergente	Mencionado no PDF como relacionado a looks, não peças
OutfitLike	Curtida em um look/outfit	MySQL	outfit_likes	Divergente	Tabela inferida mas pode estar ausente no código atual
Remix	Remixagem de um look existente	MySQL	remixes	Divergente	Liga look × look; mostra origem de inspiração
UserSavedScheme	Look salvo/favoritado por um usuário	Firestore (NoSQL) + Redis (contadores)	user_saved_schemes / favorite_count (cache)	Implementado	Tracking de favoritos; contadores em Redis para performance
AuditLog	Log de auditoria de ações (CRUD)	MySQL	audit_logs	Implementado	Conformidade e rastreamento de alterações
PieceCounters (agregado)	Contadores desnormalizados (likes, shares, etc)	Redis (cache) + MySQL (persistência eventual)	piece_counter:{piece_id} / piece_counters	Implementado	Read-through cache; atualizado por eventos do MySQL
Timeline (Feed de usuário)	Feed personalizado do usuário (looks de seguidos)	DynamoDB (series-key)	timelines (partition: user_id)	Implementado	Padrão fan-out on write; atualizado assincronamente
SearchIndex (Looks)	Índice de busca de looks por título, tag, autor	OpenSearch	looks_index	Implementado	Análise de texto; atualizado por eventos
Asset (imagem de peça/look)	Arquivo de imagem ou vídeo	S3 / Vercel Blob	bucket:fashion-ai-assets/	Implementado	Chave: {user_id}/{asset_type}/{asset_id}.{ext}

Legenda de Status

Implementado	O atributo/entidade existe na implementação atual e está preenchido no fluxo de cadastro.
Divergente	Existe no modelo, porém com outro nome, tipo ou em outra estrutura na implementação.
Ausente	Não existe na implementação atual.

Resumo da Arquitetura Poliglota

- MySQL (Relacional - Fonte da Verdade):** Armazena dados estruturados, chaves estrangeiras, e relacionamentos transacionais. Inclui Users, Brands, Markets, contadores, e logs de auditoria.
- Firestore (NoSQL - Documentos):** Armazena esquemas semi-estruturados com flexibilidade para metadados de IA. Inclui WardrobeItems, Schemes, Auditorias de IA, e Jobs de processamento.
- Redis (Cache):** Cache de leitura para contadores (likes, shares, remixes) e sessões. Alimentado por eventos do MySQL (CDC).
- DynamoDB (Series-Key):** Timeline do usuário (feed personalizado). Otimizado para leitura de série temporal com partição por user_id.
- OpenSearch (Full-Text):** Índice de busca para looks e peças. Atualizado assincronamente por eventos.

S3/Vercel Blob (Armazenamento de Objetos): Imagens, vídeos, e assets de usuário. Chaveado por user_id e tipo de asset.